



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №3

«Построение стохастическое модели в заданной предметной
области»

по курсу «Моделирование»

Студент: Аригин М.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Постановка задачи

Цель работы — построить вычислительную модель, позволяющую по набору входных показателей относить объект к одному из двух классов. Наличие контрольного признака Outcome позволяет дополнительно проверить качество полученного разбиения, однако при построении модели этот признак не используется.

Входные данные:

- файл diabetes.csv (https://disk.yandex.ru/i/M35XVBrxn09_iQ);
- числовые признаки объекта: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age.

Выходные данные:

- разбиение объектов на 2 класса (метка класса для каждого объекта);
- координаты центров классов в пространстве сниженной размерности;
- таблицы и графики для обоснования и интерпретации результатов.

2 Построение математической модели

Тип модели. Задача относится к методам многомерного статистического анализа. Рассматривается стохастическая модель, поскольку входные данные представлены выборкой случайных векторов (наборов показателей), а решение формируется на основе статистических оценок и эвристических критериев качества.

2.1 Формализация задачи

Пусть задана выборка объектов $X = \{x^{(1)}, \dots, x^{(n)}\}$, где каждый объект $x^{(k)}$ описывается p числовыми показателями. Требуется построить модель, которая сопоставляет каждому объекту метку класса $c^{(k)} \in \{1, 2\}$ так, чтобы внутри класса объекты были близки по выбранной метрике, а между классами — отличались.

2.2 Бинаризация показателей

Для каждого признака x_i вводится событие

$$A_i : x_i \leq t_i,$$

где t_i — порог бинаризации. В практической реализации используется порог по медиане $t_i = \text{median}(x_i)$ как устойчивый к выбросам вариант, формирующий близкие по размеру подгруппы.

2.3 Коэффициенты взаимосвязи

После бинаризации для каждой пары признаков (i, j) рассматриваются события $A_i : x_i \leq t_i$ и $B_j : x_j \leq t_j$. Пусть для этой пары построена таблица сопряжённости 2×2 со счётчиками

$$\begin{aligned} a &= n(A_i \cap B_j), & b &= n(A_i \cap \overline{B_j}), \\ c &= n(\overline{A_i} \cap B_j), & d &= n(\overline{A_i} \cap \overline{B_j}), \\ n &= a + b + c + d. \end{aligned}$$

В работе вычисляются несколько формальных коэффициентов взаимосвязи (без опоры на смысловую интерпретацию признаков):

- коэффициент коллигации (lift)

$$K_{ij} = \frac{P(A_i | B_j)}{P(A_i)}, \quad P(A_i) = \frac{a + b}{n}, \quad P(A_i | B_j) = \frac{a}{a + c};$$

- коэффициент контингенции Пирсона φ для 2×2 -таблицы

$$\varphi_{ij} = \frac{ad - bc}{\sqrt{(a + b)(c + d)(a + c)(b + d)}};$$

- коэффициент ассоциации Юла Q

$$Q_{ij} = \frac{ad - bc}{ad + bc};$$

- коэффициент коллигации Юла Y

$$Y_{ij} = \frac{\sqrt{ad} - \sqrt{bc}}{\sqrt{ad} + \sqrt{bc}}.$$

Для дальнейшего сокращения размерности требуется «мера близости» признаков. Для симметричных коэффициентов ($|\varphi|$, $|Q|$, $|Y|$) она задаётся модулем. Для lift используется симметризация:

$$S_{ij} = \max(K_{ij}, K_{ji}).$$

2.4 Схема модели

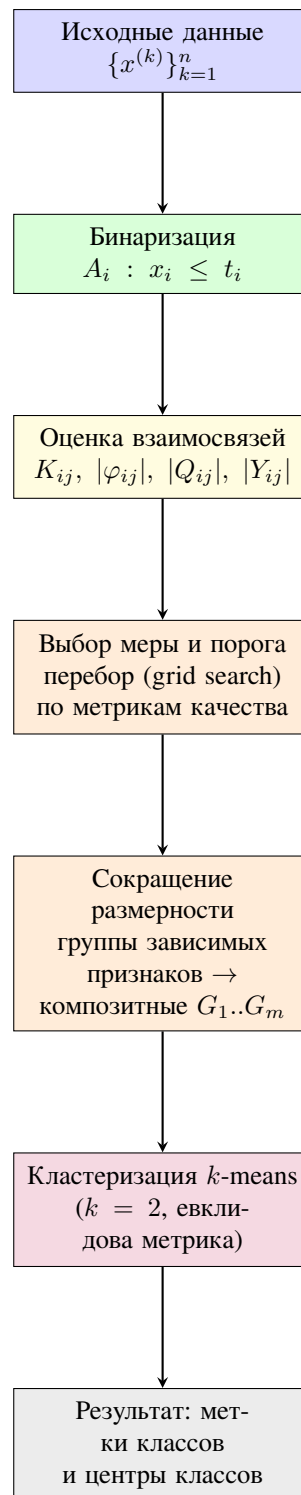


Рисунок 1 Схема вычислительной модели классификации

2.5 Сокращение размерности

Сокращение размерности выполняется без жёсткого отбрасывания признаков. По выбранной мере взаимосвязи S_{ij} и порогу S_{thr} строится граф: вершины

— признаки, ребро (i, j) проводится при $S_{ij} \geq S_{thr}$. Далее выделяются компоненты связности (группы зависимых признаков), и каждая группа заменяется композитным показателем как среднее от стандартизованных признаков группы. После построения композитных признаков выполняется стандартизация:

$$z = \frac{x - \mu}{\sigma}.$$

3 Решение вычислительных задач

3.1 Вычисление коэффициентов взаимосвязи

Для каждой пары признаков (i, j) после бинаризации строится таблица сопряжённости 2×2 с элементами (a, b, c, d) . По ней вычисляются коэффициенты K_{ij} , φ_{ij} , Q_{ij} , Y_{ij} (формулы приведены в разделе 2). Ниже приведены визуализации матриц для используемых коэффициентов.

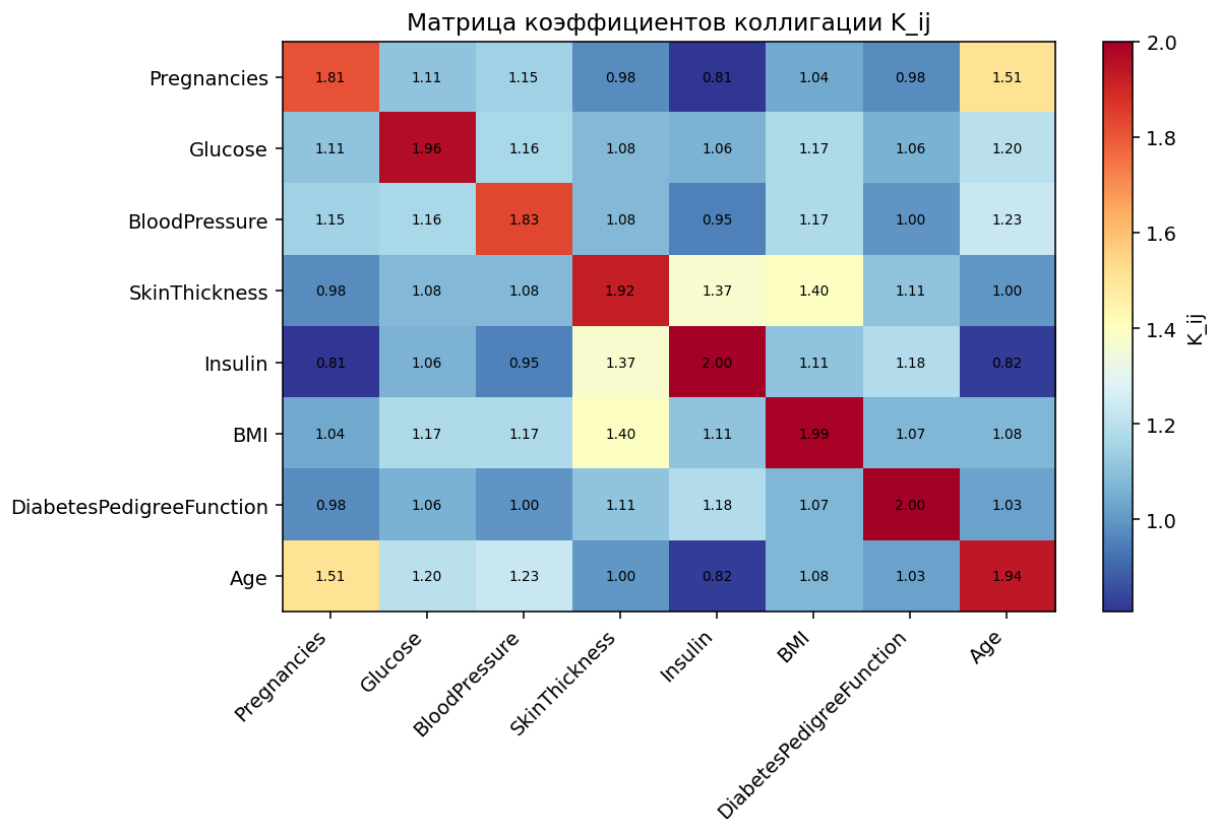


Рисунок 2 Матрица lift-коэффициента коллигации $K_{ij} = \frac{P(A_i|B_j)}{P(A_i)}$

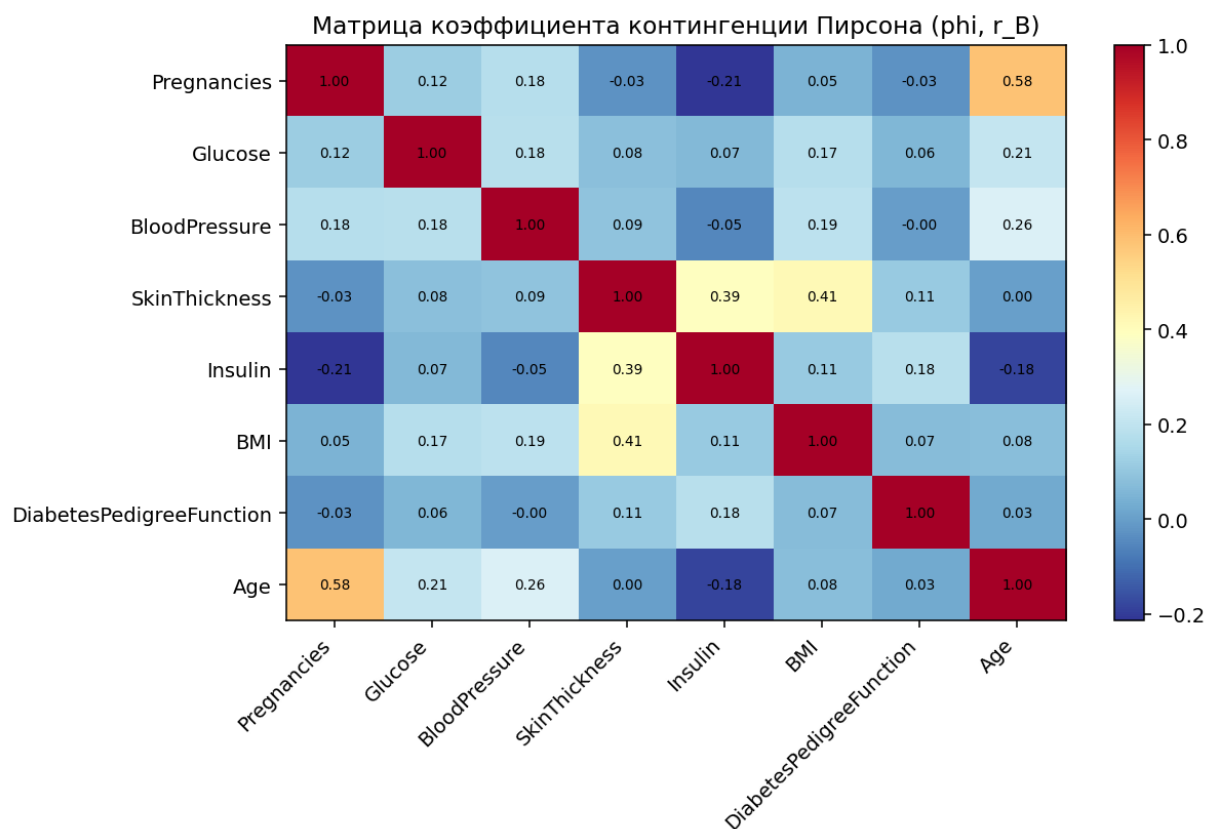


Рисунок 3 Матрица коэффициента контингенции Пирсона φ (r_B).

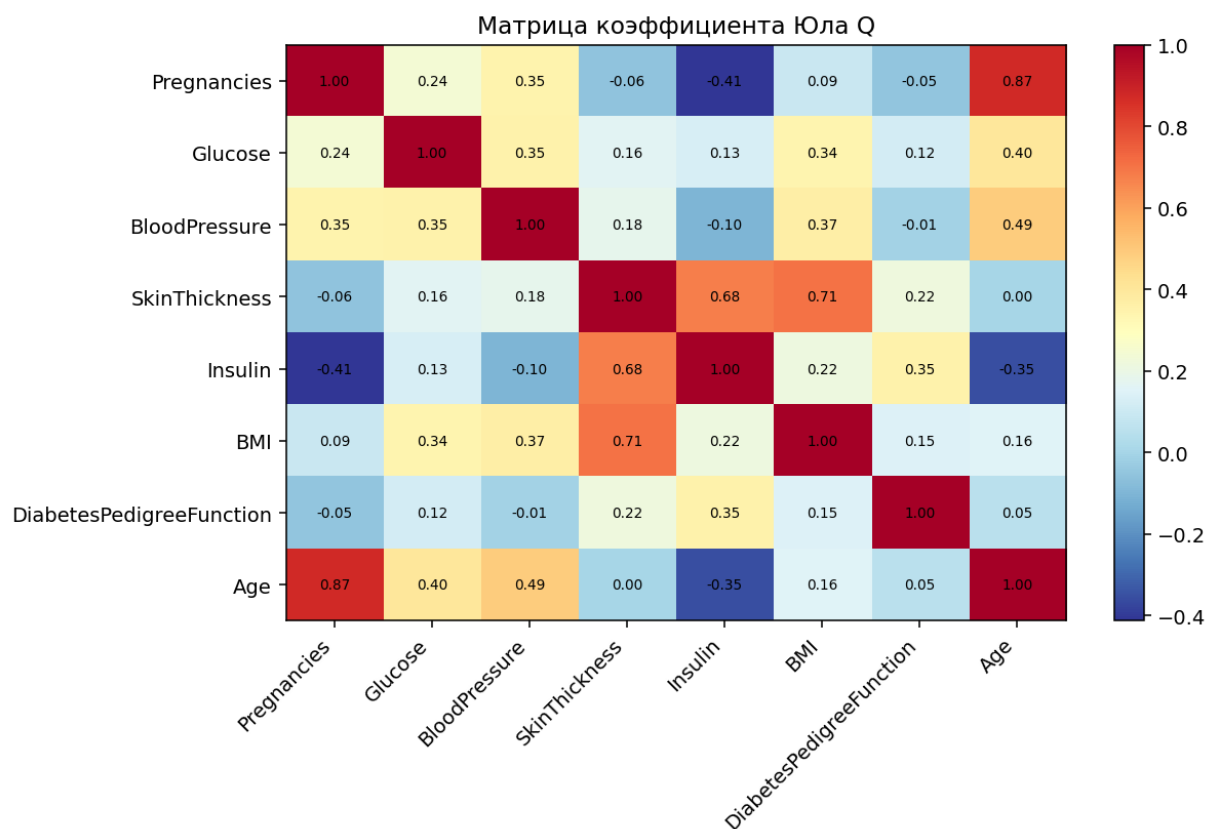


Рисунок 4 Матрица коэффициента Юла Q

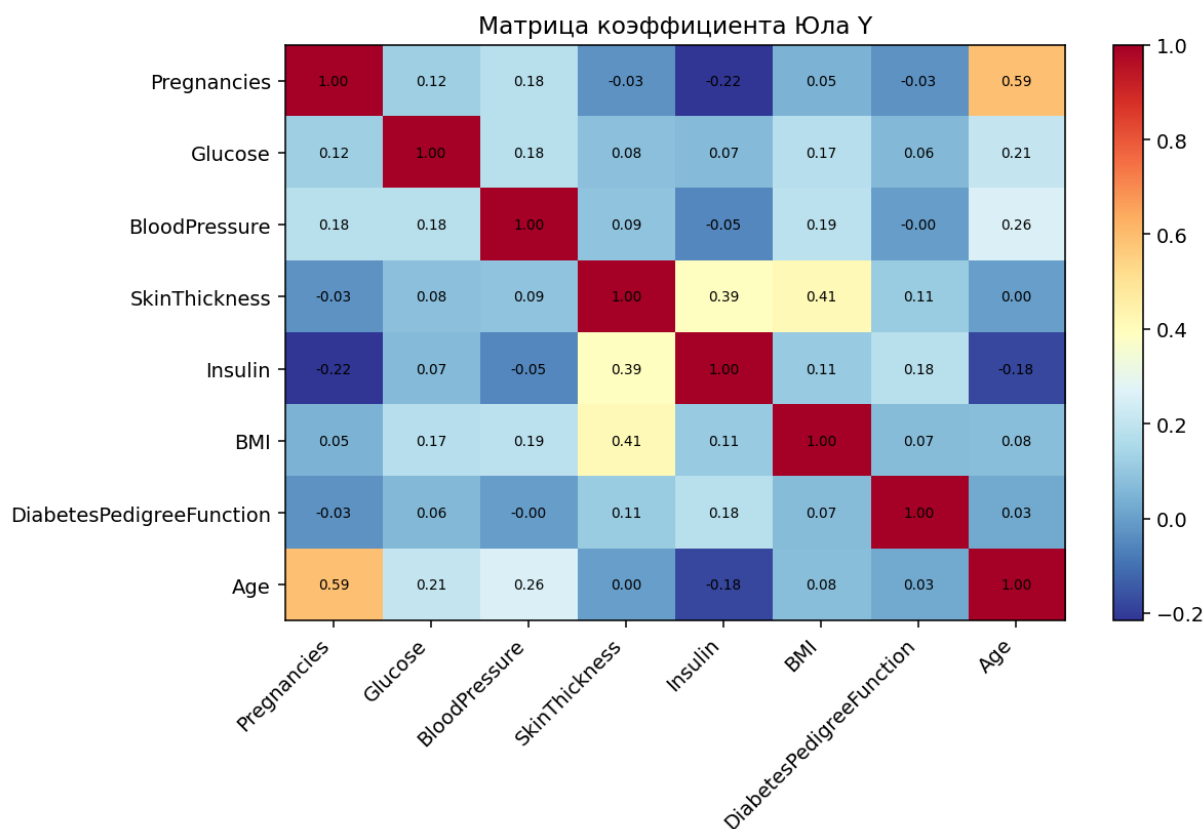


Рисунок 5 Матрица коэффициента Юла Y

3.2 Порог и построение групп зависимых признаков

По выбранным мере и порогу строятся группы зависимых признаков. Пример полученных групп приведён в табл. 1.

Пояснение к агрегированному критерию выбора. Чтобы автоматически выбрать меру взаимосвязи и порог сокращения размерности (без подбора по Outcome), используется скаляризация многокритериальной задачи через эвристический score [1]:

$$\text{score} = \text{sil} - 0.15 \cdot \ln(1 + \text{DB}) + 0.02 \cdot \ln(1 + \text{CH}) - 0.03 \cdot m + 0.10 \cdot \text{balance}.$$

Здесь sil (silhouette) и CH (Calinski–Harabasz) должны расти, а DB (Davies–Bouldin) — уменьшаться [2]; m штрафует избыточную сложность (число композитных признаков), $\text{balance} = \min(n_1, n_2) / \max(n_1, n_2)$ поощряет невырожденное разбиение по размерам кластеров. Логарифмирование $\ln(1 + x)$ применяется, чтобы привести метрики разного масштаба к сопоставимому влиянию

на сумму (например, СН может быть на порядки больше sil). Весовые коэффициенты являются эвристическими и задают компромисс между качеством кластеризации, устойчивостью и простотой модели.

Замечание о мультиколлинеарности. В мультиколлинеарных пространствах (сильно коррелированные признаки) при построении композитных показателей меняется геометрия данных и масштабы расстояний; поэтому оптимизация внутренних метрик и простоты модели может давать предпочтение меньшему m . Важно, что **score оптимизирует другое** — внутреннюю геометрию кластеров и простоту (через штраф за размерность), поэтому он может не совпасть с внешним критерием по Outcome.

Group Features	
G1	Age, BloodPressure, Pregnancies
G2	BMI, Insulin, SkinThickness
G3	Glucose
G4	DiabetesPedigreeFunction

Таблица 1: Группы признаков после сокращения размерности

3.3 Кластеризация методом k -means

Кластеризация выполняется в пространстве композитных признаков $(G_1, \dots, G_m)$ по евклидовой метрике. Начальные центры выбираются как первая точка и наиболее удалённая от неё, затем итеративно выполняются шаги:

1. назначить каждой точке ближайший центр;
2. пересчитать центры как среднее по точкам кластера;
3. повторять до стабилизации центров.

Размеры кластеров:

Cluster Count	
1	482
2	286

Таблица 2: Количество объектов в кластерах

Центры кластеров (в композитных стандартизированных координатах):

	G1	G2	G3	G4
Cluster				
1	-0.1753	-0.2297	-0.5629	-0.2712
2	0.2955	0.3871	0.9487	0.4571

Таблица 3: Центры кластеров в сниженной размерности

4 Практическая реализация

Listing 1: Листинг программной реализации (сокращённая версия для отчёта).

```

1  import os
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5
6  def contingency_2x2(a_event: pd.Series, b_event: pd.Series) -> tuple[int, int, int, int]:
7      a_event = a_event.astype(bool)
8      b_event = b_event.astype(bool)
9      a = int((a_event & b_event).sum())
10     b = int((a_event & ~b_event).sum())
11     c = int((~a_event & b_event).sum())
12     d = int((~a_event & ~b_event).sum())
13     return a, b, c, d
14
15  def colligation_k(a: int, b: int, c: int, d: int) -> float:
16     n = a + b + c + d
17     p_a = (a + b) / n if n else 0.0
18     p_a_given_b = a / (a + c) if (a + c) else 0.0
19     return (p_a_given_b / p_a) if p_a else 0.0
20
21  def phi_coef(a: int, b: int, c: int, d: int) -> float:
22     denom = (a + b) * (c + d) * (a + c) * (b + d)
23     return (a * d - b * c) / np.sqrt(denom) if denom else 0.0
24
25  def yule_q(a: int, b: int, c: int, d: int) -> float:
26     denom = a * d + b * c
27     return (a * d - b * c) / denom if denom else 0.0
28
29  def yule_y(a: int, b: int, c: int, d: int) -> float:
30     ad = a * d
31     bc = b * c
32     denom = np.sqrt(ad) + np.sqrt(bc)

```

```

33     return (np.sqrt(ad) - np.sqrt(bc)) / denom if denom else 0.0
34
35 def zscore(X: np.ndarray) -> np.ndarray:
36     mu = X.mean(axis=0)
37     sigma = X.std(axis=0)
38     sigma[sigma == 0] = 1.0
39     return (X - mu) / sigma
40
41 def mst_groups(sim: pd.DataFrame, target_m: int) -> tuple[list[list[str]], float]:
42     cols = list(sim.index)
43     p = len(cols)
44     if target_m < 1 or target_m > p:
45         raise ValueError("target_m out of range")
46
47     edges: list[tuple[float, int, int]] = []
48     for i in range(p):
49         for j in range(i + 1, p):
50             w = max(float(sim.iloc[i, j]), float(sim.iloc[j, i]))
51             edges.append((w, i, j))
52     edges.sort(reverse=True, key=lambda t: t[0])
53
54     parent = list(range(p))
55     size = [1] * p
56
57     def find(x: int) -> int:
58         while parent[x] != x:
59             parent[x] = parent[parent[x]]
60             x = parent[x]
61         return x
62
63     def union(a: int, b: int) -> bool:
64         ra, rb = find(a), find(b)
65         if ra == rb:
66             return False
67         if size[ra] < size[rb]:
68             ra, rb = rb, ra
69         parent[rb] = ra
70         size[ra] += size[rb]
71         return True
72
73     comps = p
74     last_w = float("nan")
75     for w, i, j in edges:
76         if comps <= target_m:
77             break
78         if union(i, j):
79             comps -= 1

```

```

80         last_w = w
81
82     groups: dict[int, list[str]] = {}
83     for i, name in enumerate(cols):
84         r = find(i)
85         groups.setdefault(r, []).append(name)
86
87     components = [sorted(v) for v in groups.values()]
88     components = sorted(components, key=len, reverse=True)
89     return components, last_w
90
91 def make_composites(df: pd.DataFrame, features: list[str], components: list[list[str]]) ->
92     ↪ tuple[np.ndarray, list[str]]:
93     X = df[features].to_numpy(dtype=float)
94     Z = zscore(X)
95     comp_names: list[str] = []
96     Z_comp: list[np.ndarray] = []
97     for gi, comp in enumerate(components, start=1):
98         idxs = [features.index(c) for c in comp]
99         Z_comp.append(Z[:, idxs].mean(axis=1))
100         comp_names.append(f"G{gi}")
101     Z_red = np.vstack(Z_comp).T
102     return Z_red, comp_names
103
104 def kmeans2_manual(Xr: np.ndarray, max_iter: int = 100, tol: float = 1e-6) -> tuple[np.ndarray,
105     ↪ np.ndarray]:
106     c1 = Xr[0].copy()
107     d2 = np.sum((Xr - c1) ** 2, axis=1)
108     c2 = Xr[np.argmax(d2)].copy()
109     centers = np.vstack([c1, c2])
110
111     for _ in range(max_iter):
112         dist1 = np.sqrt(np.sum((Xr - centers[0]) ** 2, axis=1))
113         dist2 = np.sqrt(np.sum((Xr - centers[1]) ** 2, axis=1))
114         labels = (dist2 < dist1).astype(int)
115
116         new_centers = centers.copy()
117         for k in (0, 1):
118             pts = Xr[labels == k]
119             if len(pts) > 0:
120                 new_centers[k] = pts.mean(axis=0)
121
122         if np.max(np.abs(new_centers - centers)) < tol:
123             centers = new_centers
124             break
125     centers = new_centers

```

```

125     return labels, centers
126
127 def pairwise_dist(X: np.ndarray) -> np.ndarray:
128     diff = X[:, None, :] - X[None, :, :]
129     return np.sqrt(np.sum(diff * diff, axis=2))
130
131 def silhouette_score(X: np.ndarray, labels: np.ndarray) -> float:
132     D = pairwise_dist(X)
133     n = len(labels)
134     s = np.zeros(n)
135     for i in range(n):
136         same = labels == labels[i]
137         other = ~same
138         a = D[i, same].sum() / (same.sum() - 1) if same.sum() > 1 else 0.0
139         b = D[i, other].mean() if other.sum() > 0 else 0.0
140         denom = max(a, b)
141         s[i] = (b - a) / denom if denom > 0 else 0.0
142     return float(s.mean())
143
144 def davies_bouldin(X: np.ndarray, labels: np.ndarray, centers: np.ndarray) -> float:
145     db = 0.0
146     for k in (0, 1):
147         pts_k = X[labels == k]
148         if len(pts_k) == 0:
149             return float("inf")
150         S_k = np.mean(np.sqrt(np.sum((pts_k - centers[k]) ** 2, axis=1)))
151         worst = -1.0
152         for l in (0, 1):
153             if l == k:
154                 continue
155             pts_l = X[labels == l]
156             if len(pts_l) == 0:
157                 return float("inf")
158             S_l = np.mean(np.sqrt(np.sum((pts_l - centers[l]) ** 2, axis=1)))
159             M = np.sqrt(np.sum((centers[k] - centers[l]) ** 2))
160             R = (S_k + S_l) / M if M > 0 else float("inf")
161             worst = max(worst, R)
162         db += worst
163     return float(db / 2)
164
165 def calinski_harabasz(X: np.ndarray, labels: np.ndarray, centers: np.ndarray) -> float:
166     n = len(labels)
167     overall = X.mean(axis=0)
168     W = 0.0
169     for k in (0, 1):
170         pts = X[labels == k]
171         W += np.sum((pts - centers[k]) ** 2)

```

```

172 B = 0.0
173 for k in (0, 1):
174     nk = (labels == k).sum()
175     B += nk * np.sum((centers[k] - overall) ** 2)
176 if W == 0 or n <= 2:
177     return 0.0
178 return float((B / 1) / (W / (n - 2)))
179
180 def external_metrics_best_mapping(y: np.ndarray, labels01: np.ndarray) -> dict[str, float]:
181     def metrics(pred: np.ndarray) -> dict[str, float]:
182         tn = int(((pred == 0) & (y == 0)).sum())
183         fp = int(((pred == 1) & (y == 0)).sum())
184         fn = int(((pred == 0) & (y == 1)).sum())
185         tp = int(((pred == 1) & (y == 1)).sum())
186         n = len(y)
187
188         acc = (tp + tn) / n if n else 0.0
189
190         prec1 = tp / (tp + fp) if (tp + fp) else 0.0
191         rec1 = tp / (tp + fn) if (tp + fn) else 0.0
192         f11 = (2 * prec1 * rec1 / (prec1 + rec1)) if (prec1 + rec1) else 0.0
193
194         prec0 = tn / (tn + fn) if (tn + fn) else 0.0
195         rec0 = tn / (tn + fp) if (tn + fp) else 0.0
196         f10 = (2 * prec0 * rec0 / (prec0 + rec0)) if (prec0 + rec0) else 0.0
197
198         f1_macro = 0.5 * (f10 + f11)
199         bal_acc = 0.5 * (rec0 + rec1)
200
201         return {
202             "Acc": float(acc),
203             "BalAcc": float(bal_acc),
204             "F1-macro": float(f1_macro),
205             "Prec(1)": float(prec1),
206             "Rec(1)": float(rec1),
207             "F1(1)": float(f11),
208         }
209
210     m0 = metrics(labels01.astype(int))
211     m1 = metrics((1 - labels01).astype(int))
212     return m0 if m0["F1-macro"] >= m1["F1-macro"] else m1
213
214 def save_heatmap(mat: pd.DataFrame, title: str, fname: str, cmap: str = "RdYlBu_r") -> None:
215     plt.figure(figsize=(9, 6))
216     X = mat.to_numpy(dtype=float)
217     plt.imshow(X, cmap=cmap, aspect="auto")
218     plt.xticks(range(X.shape[1]), mat.columns, rotation=45, ha="right")

```

```

219 plt.yticks(range(X.shape[0]), mat.index)
220 plt.colorbar()
221 plt.title(title)
222 plt.tight_layout()
223 plt.savefig(fname, bbox_inches="tight")
224 plt.close()
225
226 def main() -> None:
227     os.makedirs("images", exist_ok=True)
228     os.makedirs("tables", exist_ok=True)
229
230     df = pd.read_csv("diabetes.csv")
231     features = [c for c in df.columns if c != "Outcome"]
232
233     thr = {c: float(df[c].median()) for c in features}
234     Xbin = pd.DataFrame({c: (df[c] <= thr[c]) for c in features})
235
236     K = pd.DataFrame(index=features, columns=features, dtype=float)
237     PHI = pd.DataFrame(index=features, columns=features, dtype=float)
238     Q = pd.DataFrame(index=features, columns=features, dtype=float)
239     Y = pd.DataFrame(index=features, columns=features, dtype=float)
240
241     for i in features:
242         for j in features:
243             a, b, c, d = contingency_2x2(Xbin[i], Xbin[j])
244             K.loc[i, j] = colligation_k(a, b, c, d)
245             PHI.loc[i, j] = phi_coef(a, b, c, d)
246             Q.loc[i, j] = yule_q(a, b, c, d)
247             Y.loc[i, j] = yule_y(a, b, c, d)
248
249     save_heatmap(K, "Матрица коэффициента коллигации K_ij",
250                 ↪ "images/colligation_K_heatmap.png")
251     save_heatmap(PHI, "Матрица коэффициента контингенции Пирсона (phi, r_B)",
252                 ↪ "images/phi_heatmap.png")
253     save_heatmap(Q, "Матрица коэффициента Юла Q", "images/yule_Q_heatmap.png")
254     save_heatmap(Y, "Матрица коэффициента Юла Y", "images/yule_Y_heatmap.png")
255
256     SIMS = {"abs_Y": Y.abs(), "abs_Q": Q.abs(), "abs_phi": PHI.abs(), "K": K}
257
258     rows = []
259     y_true = df["Outcome"].to_numpy(dtype=int)
260
261     for name, sim in SIMS.items():
262         for m in range(2, len(features) + 1):
263             comps, thr_mst = mst_groups(sim, target_m=m)
264             Xr, comp_names = make_composites(df, features, comps)
265             labels, centers = kmeans2_manual(Xr)

```

```

264
265     n1 = int((labels == 0).sum())
266     n2 = int((labels == 1).sum())
267     balance = min(n1, n2) / max(n1, n2)
268
269     sil = silhouette_score(Xr, labels)
270     db = davies_bouldin(Xr, labels, centers)
271     ch = calinski_harabasz(Xr, labels, centers)
272     score = sil - 0.15 * np.log1p(db) + 0.02 * np.log1p(ch) - 0.03 * m + 0.10 * balance
273
274     ext = external_metrics_best_mapping(y_true, labels.astype(int))
275
276     rows.append(
277         {
278             "measure": name,
279             "thr": float(thr_mst),
280             "m": int(m),
281             "balance": float(balance),
282             "silhouette": float(sil),
283             "Davies--Bouldin": float(db),
284             "Calinski--Harabasz": float(ch),
285             "score": float(score),
286             **ext,
287             "components": comps,
288             "comp_names": comp_names,
289             "centers": centers,
290         }
291     )
292
293     results = pd.DataFrame(rows)
294
295     def best_for(measure: str, m: int) -> pd.Series:
296         sub = results[(results["measure"] == measure) & (results["m"] == m)].copy()
297         return sub.sort_values(["F1-macro", "BalAcc", "score"], ascending=False).iloc[0]
298
299     measures_order = ["abs_Y", "abs_Q", "abs_phi", "K"]
300     rows28 = []
301     for meas in measures_order:
302         for mm in range(2, 9):
303             r = best_for(meas, mm)
304             rows28.append(
305                 {
306                     "measure": r["measure"].replace("_", r"\_"),
307                     "thr": r["thr"],
308                     "m": r["m"],
309                     "balance": r["balance"],
310                     "silhouette": r["silhouette"],

```

```

311         "Davies--Bouldin": r["Davies--Bouldin"],
312         "Calinski--Harabasz": r["Calinski--Harabasz"],
313         "score": r["score"],
314         "Acc": r["Acc"],
315         "BalAcc": r["BalAcc"],
316         "F1-macro": r["F1-macro"],
317         "Prec(1)": r["Prec(1)"],
318         "Rec(1)": r["Rec(1)"],
319         "F1(1)": r["F1(1)"],
320     }
321 )
322
323 tab4 = pd.DataFrame(rows28)
324 tab4.to_latex("tables/threshold_search.tex", index=False, escape=False, float_format=lambda x:
325     ↪ f"{x:.4f}")
326
327 acc_by_m = (
328     results.groupby("m", as_index=False)
329     .agg(best_accuracy=("Acc", "max"), best_balanced_accuracy=("BalAcc", "max"),
330     ↪ best_f1_macro=("F1-macro", "max"), best_score=("score", "max"))
331     .sort_values("m")
332 )
333 acc_by_m.to_latex("tables/accuracy_by_m.tex", index=False, escape=True,
334     ↪ float_format=lambda x: f"{x:.4f}")
335
336 best_macro = results.sort_values(["F1-macro", "BalAcc", "score"], ascending=False).iloc[0]
337 comps = best_macro["components"]
338 Xr, comp_names = make_composites(df, features, comps)
339 labels, centers = kmeans2_manual(Xr)
340
341 out = df.copy()
342 out["Cluster"] = labels + 1
343 out.to_csv("diabetes_kmeans_clusters_reduced.csv", index=False)
344
345 with open("tables/accuracy.tex", "w", encoding="utf-8") as f:
346     f.write("\\\\begin{tabular}{lr}\\\\n\\\\\\\\toprule\\\\nМетрика & Значение\\\\n\\\\\\\\midrule\\\\n")
347     ↪ f.write(f"Acc & {float(best_macro['Acc']):.6f}\\\\n")
348     f.write(f"BalAcc & {float(best_macro['BalAcc']):.6f}\\\\n")
349     f.write(f"F1-macro & {float(best_macro['F1-macro']):.6f}\\\\n")
350     f.write(f"Prec(1) & {float(best_macro['Prec(1)']):.6f}\\\\n")
351     f.write(f"Rec(1) & {float(best_macro['Rec(1)']):.6f}\\\\n")
352     f.write(f"F1(1) & {float(best_macro['F1(1)']):.6f}\\\\n")
353     f.write("\\\\bottomrule\\\\n\\\\end{tabular}\\\\n")
354
355 if __name__ == "__main__":
356     main()

```

5 Результаты

Получено разбиение объектов на 2 класса. Ниже приведены результаты вычисления матриц взаимосвязей, выбора меры/порога для сокращения размерности и визуализация кластеризации.

5.1 Выбор меры взаимосвязи и порога

Мера взаимосвязи и порог для построения групп зависимых признаков выбираются автоматически перебором кандидатов: $S_{ij} \in \{\max(K_{ij}, K_{ji}), |\varphi_{ij}|, |Q_{ij}|, |Y_{ij}|\}$ и S_{thr} из заданной сетки. Каждый кандидат оценивается по внутренним метрикам качества кластеризации и балансу кластеров; при этом рассматриваются варианты размерности от $m = 2$ до $m = p$ (где p — число исходных признаков).

Замечание о выборе. Внутренние метрики (silhouette, Davies–Bouldin, Calinski–Harabasz) оценивают геометрию кластеров без учителя, поэтому «лучший» по ним вариант может не совпадать с разбиением, наиболее согласованным с контрольной меткой Outcome. Так как классы Outcome несбалансированы, метрика ассюрасу может быть обманчивой; поэтому для внешней контрольной оценки используются balanced ассюрасу и F1 (macro-F1). При расчёте внешних метрик выбирается лучшее сопоставление меток кластеров и Outcome.

Выбранный вариант для итогового результата (по максимуму macro-F1): $S_{ij} = |Y_{ij}|$, $S_{thr} = 0.25$, итоговая размерность $m = 4$, $F1_{macro} = 0.7261$, $BalAcc = 0.7298$.

Сравнение с вариантом, выбранным по score: при $S_{ij} = |Y_{ij}|$, $S_{thr} = 0.20$, $m = 2$ получаем $F1_{macro} = 0.5842$, $BalAcc = 0.5831$.

measure	thr	m	balance	silhouette	Davies–Bouldin	Calinski–Harabasz	score	Acc	BalAcc	F1-macro	Prec(1)	Rec(1)	F1(1)
abs_Y	0.182476	2	0.333333	0.504632	0.815156	749.578001	0.520956	0.651042	0.583104	0.584160	0.500000	0.358209	0.417391
abs_Y	0.215999	3	0.644540	0.330783	1.297595	358.524427	0.298154	0.730469	0.717687	0.711070	0.601329	0.675373	0.636204
abs_Y	0.262367	4	0.593361	0.273669	1.546834	265.806661	0.184508	0.747396	0.729821	0.726132	0.629371	0.671642	0.649819
abs_Y	0.388719	5	0.777778	0.218669	1.757349	203.640304	0.100731	0.705729	0.705612	0.691669	0.562500	0.705224	0.625828
abs_Y	0.414552	6	0.560976	0.212648	1.860686	180.541893	0.035116	0.721354	0.696836	0.695446	0.597826	0.615672	0.606618
abs_Y	0.414552	7	0.648069	0.189604	1.881788	178.735806	-0.010521	0.658854	0.639313	0.634565	0.509934	0.574627	0.540351
abs_Y	0.587239	8	0.600000	0.170813	2.086321	146.949016	-0.078296	0.677083	0.653313	0.650448	0.534722	0.574627	0.553957
abs_Q	0.352858	2	0.333333	0.504632	0.815156	749.578001	0.520956	0.651042	0.583104	0.584160	0.500000	0.358209	0.417391
abs_Q	0.412741	3	0.644540	0.330783	1.297595	358.524427	0.298154	0.730469	0.717687	0.711070	0.601329	0.675373	0.636204
abs_Q	0.490940	4	0.593361	0.273669	1.546834	265.806661	0.184508	0.747396	0.729821	0.726132	0.629371	0.671642	0.649819
abs_Q	0.490940	5	0.777778	0.218669	1.757349	203.640304	0.100731	0.705729	0.705612	0.691669	0.562500	0.705224	0.625828
abs_Q	0.707515	6	0.560976	0.212648	1.860686	180.541893	0.035116	0.721354	0.696836	0.695446	0.597826	0.615672	0.606618
abs_Q	0.873315	7	0.648069	0.189604	1.881788	178.735806	-0.010521	0.658854	0.639313	0.634565	0.509934	0.574627	0.540351
abs_Q	0.873315	8	0.600000	0.170813	2.086321	146.949016	-0.078296	0.677083	0.653313	0.650448	0.534722	0.574627	0.553957
abs_phi	0.190425	2	0.333333	0.504632	0.815156	749.578001	0.520956	0.651042	0.583104	0.584160	0.500000	0.358209	0.417391
abs_phi	0.214710	3	0.644540	0.330783	1.297595	358.524427	0.298154	0.730469	0.717687	0.711070	0.601329	0.675373	0.636204
abs_phi	0.261388	4	0.593361	0.273669	1.546834	265.806661	0.184508	0.747396	0.729821	0.726132	0.629371	0.671642	0.649819
abs_phi	0.388317	5	0.777778	0.218669	1.757349	203.640304	0.100731	0.705729	0.705612	0.691669	0.562500	0.705224	0.625828
abs_phi	0.414181	6	0.560976	0.212648	1.860686	180.541893	0.035116	0.721354	0.696836	0.695446	0.597826	0.615672	0.606618
abs_phi	0.583536	7	0.648069	0.189604	1.881788	178.735806	-0.010521	0.658854	0.639313	0.634565	0.509934	0.574627	0.540351
abs_phi	0.583536	8	0.600000	0.170813	2.086321	146.949016	-0.078296	0.677083	0.653313	0.650448	0.534722	0.574627	0.553957
K	1.172889	2	0.864078	0.347522	1.146608	443.307302	0.381277	0.695312	0.702806	0.684211	0.547753	0.727612	0.625000
K	1.182292	3	0.345009	0.363328	1.192755	377.421042	0.308775	0.662760	0.598164	0.600590	0.522843	0.384328	0.443011
K	1.231214	4	0.593361	0.273669	1.546834	265.806661	0.184508	0.747396	0.729821	0.726132	0.629371	0.671642	0.649819
K	1.373434	5	0.777778	0.218669	1.757349	203.640304	0.100731	0.705729	0.705612	0.691669	0.562500	0.705224	0.625828
K	1.373434	6	0.560976	0.212648	1.860686	180.541893	0.035116	0.721354	0.696836	0.695446	0.597826	0.615672	0.606618
K	1.509434	7	0.648069	0.189604	1.881788	178.735806	-0.010521	0.658854	0.639313	0.634565	0.509934	0.574627	0.540351
K	1.509434	8	0.600000	0.170813	2.086321	146.949016	-0.078296	0.677083	0.653313	0.650448	0.534722	0.574627	0.553957

Таблица 4: Сравнение мер взаимосвязи и порогов по метрикам качества кластеризации и внешним контрольным метрикам (Acc, BalAcc, F1-macro)

m	best_accuracy	best_balanced_accuracy	best_f1_macro	best_score
2	0.6953	0.7028	0.6842	0.5210
3	0.7305	0.7177	0.7111	0.3088
4	0.7474	0.7298	0.7261	0.1845
5	0.7057	0.7056	0.6917	0.1007
6	0.7214	0.6968	0.6954	0.0351
7	0.6589	0.6393	0.6346	-0.0105
8	0.6771	0.6533	0.6504	-0.0783

Таблица 5: Лучшие внешние метрики по Outcome в зависимости от числа ком-
позитных признаков m

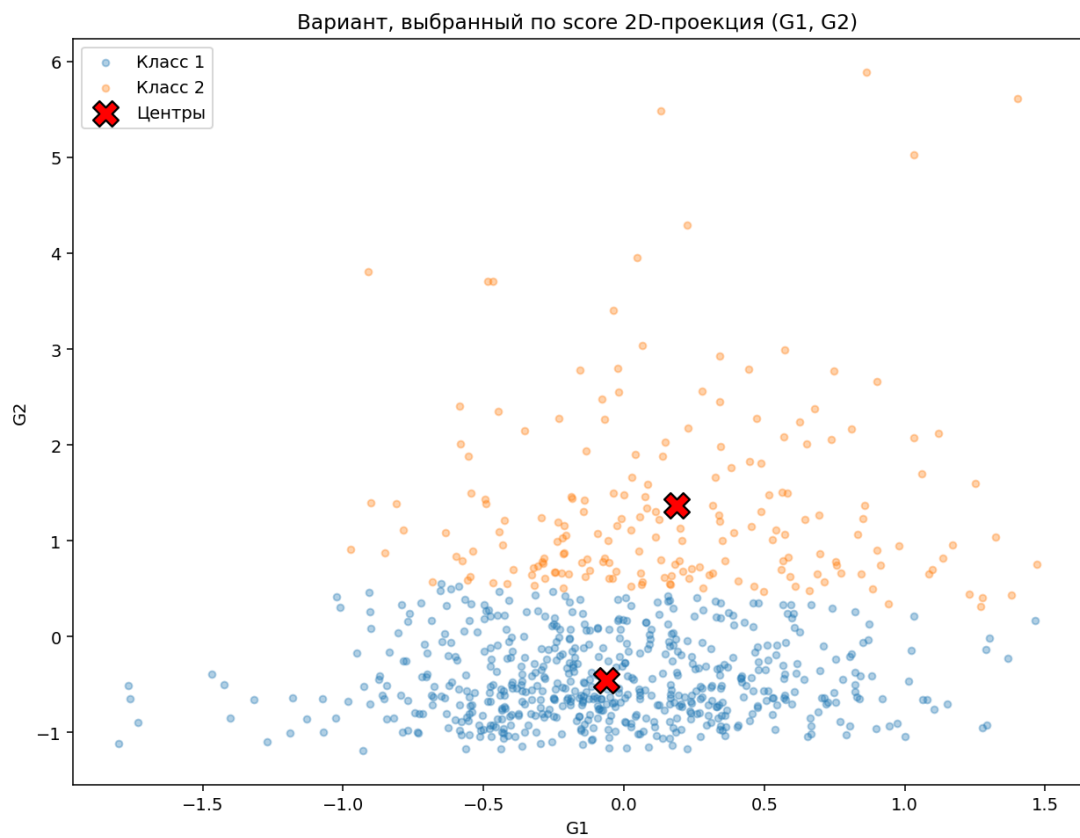


Рисунок 6 Разбиение k -means в 2D-проекции (вариант, выбранный по внутреннему score)

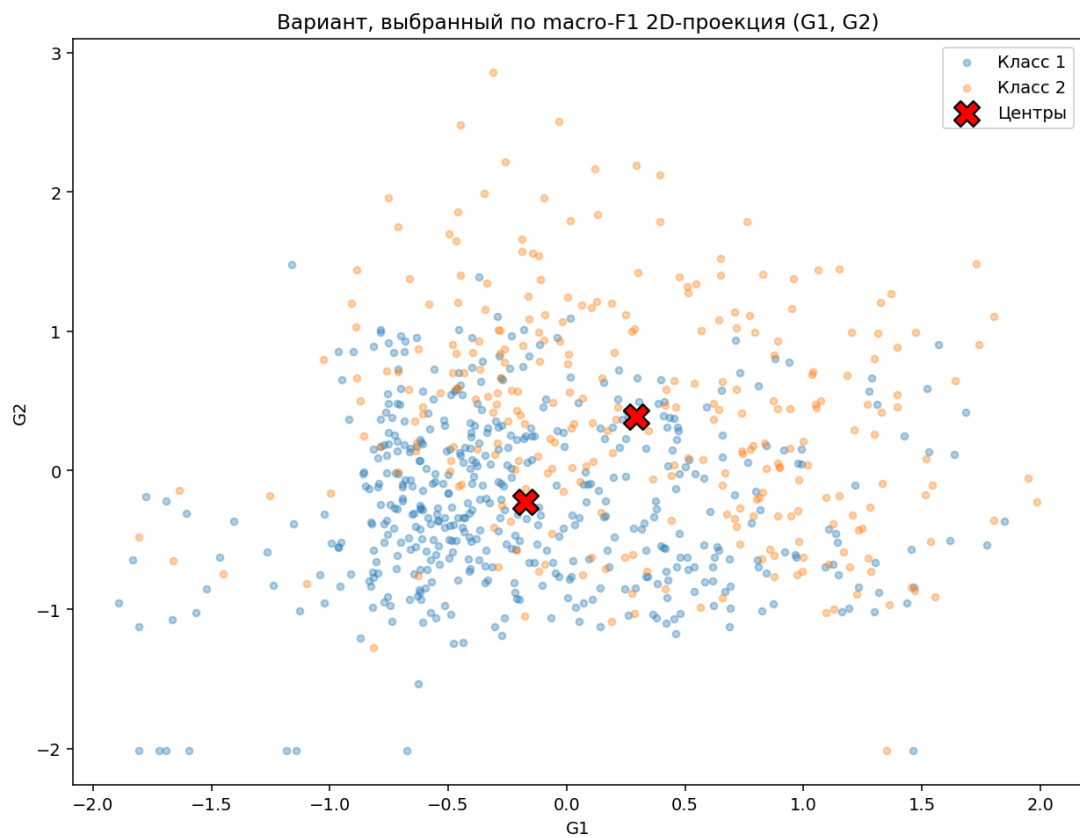


Рисунок 7 Разбиение k -means в 2D-проекции (вариант, выбранный по максимуму макро-F1 по Outcome)

Вариант, выбранный по macro-F1 3D-проекция (G1, G2, G3)

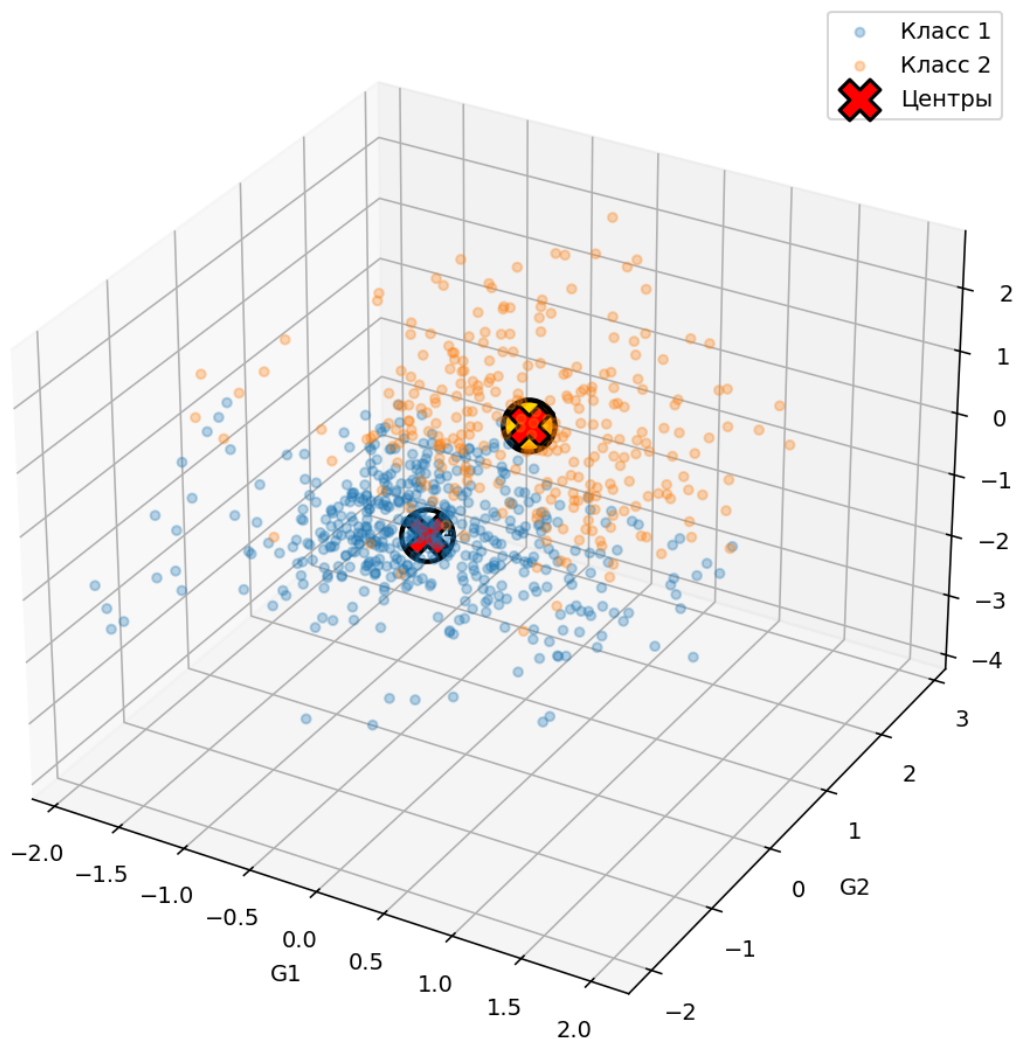


Рисунок 8 Разбиение k -means в 3D-проекции (вариант, выбранный по максимуму macro-F1; G1, G2, G3)

6 Анализ моделей и проверка качества

Качество кластеризации оценивается как внутренними метриками, так и внешней проверкой (без использования Outcome при построении модели).

6.1 Внутренние метрики качества

Для выбранного разбиения вычисляются стандартные внутренние метрики:

- средний коэффициент силуэта (silhouette) — чем больше, тем лучше;
- индекс Дэвиса–Болдина (Davies–Bouldin) — чем меньше, тем лучше;

- индекс Калински–Харабаса (Calinski–Harabasz) — чем больше, тем лучше.

Согласно теории кластерного анализа, решение является принципиально неоднозначным: не существует единственного «наилучшего» критерия качества кластеризации, результат зависит от выбранной метрики расстояния и стандартизации признаков, а число кластеров часто задаётся исследователем. Поэтому в работе используются как внутренние метрики качества, так и внешняя контрольная оценка по Outcome (только для проверки результата, без участия в построении модели).

6.2 Внешняя проверка по Outcome

Столбец Outcome не использовался при построении модели и кластеризации. Однако его можно применить для внешней проверки согласованности разбиения: сравнивается распределение значений Outcome в полученных классах и вычисляются внешние контрольные метрики. Так как классы Outcome несбалансированы, для контроля используются BalAcc и F1 (macro-F1).

Метрика	Значение
Acc	0.747396
BalAcc	0.729821
F1-macro	0.726132
Prec(1)	0.629371
Rec(1)	0.671642
F1(1)	0.649819

Таблица 6: Внешние контрольные метрики по Outcome (лучшее сопоставление меток кластеров и Outcome)

7 Выводы

В ходе работы построена стохастическая вычислительная модель отнесения объектов к двум классам по входным данным из diabetes.csv. Получена полная цепочка вычислительных процедур:

- выполнена бинаризация каждого числового признака по устойчивому порогу $t_i = \text{median}(x_i)$, что обеспечивает формирование сопоставимых по объёму подвыборок и снижает влияние выбросов;
- рассчитаны матрицы взаимосвязи признаков по нескольким формальным коэффициентам (K_{ij} , φ , Юла Q , Юла Y), что позволяет не фиксировать заранее единственную меру зависимости;
- реализован алгоритм выбора меры взаимосвязи и порога сокращения размерности: перебираются кандидаты, строятся группы зависимых признаков и композитные показатели $G_1, \dots, G_m$, после чего вариант оценивается по внутренним метрикам качества кластеризации и балансу кластеров для размерностей от $m = 2$ до $m = p$ (где p — число исходных признаков);
- выполнена кластеризация методом k -means в пространстве композитных признаков с получением центров кластеров и меток классов;
- проведена формальная интерпретация полученных классов через сравнение статистик исходных признаков внутри каждого класса.

Для контроля качества использована внешняя проверка по признаку Outcome. Из-за дисбаланса классов основная контрольная метрика — macro-F1 (и balanced accuracy). При снятии ограничения на m оказалось, что вариант, оптимальный по внутреннему score, даёт меньшие внешние метрики (например, при $m = 2$ $F1_{\text{macro}} \approx 0.584$), тогда как при $m = 4$ достигаются существенно лучшие значения ($F1_{\text{macro}} \approx 0.726$). Поэтому в итоговых результатах выбран вариант, максимизирующий macro-F1, а внутренние метрики приведены как характеристика структуры кластеров.

Список литературы

- [1] Подиновский В.В., Потапов М.А. Метод взвешенной суммы критериев в анализе многокритериальных решений: *pro et contra* // *Бизнес-информатика*. 2013.

- [2] *scikit-learn: руководство пользователя (русский перевод)*. Раздел «Кластеризация» и метрики качества (silhouette, Davies–Bouldin, Calinski–Harabasz). Дата обращения: 31.03.2026. URL: <https://scikit-learn.ru/stable/modules/clustering.html>.